



UNIVERSIDAD
CATÓLICA DE
TEMUCO

INGENIERÍA CIVIL
EN INFORMÁTICA
FACULTAD DE INGENIERÍA

UNIVERSIDAD CATÓLICA DE TEMUCO

Facultad de Ingeniería

Tarea

Análisis léxico de un subconjunto de Prolog

Asignatura: INFO1148

Integrantes: Benavides Monsálvez, Patricio (Jefe de grupo)
Escares Ortiz, Eduardo
Ferrer Retamal, Christian

Carrera: Ingeniería Civil en Informática

Docente: Marcos Levano

Fecha de entrega: 22 de Septiembre de 2026

Semestre: Segundo semestre 2026

Índice

1. Introducción	2
2. Objetivos	2
2.1. Objetivo general	2
2.2. Objetivos específicos	2
3. Alcance del analizador	3
4. Especificación léxica	3
5. Diseño de los autómatas	4
5.1. AFD para ATOMO y VARIABLE	4
5.2. AFD para números	6
5.3. AFD de operadores con prefijo común	6
5.4. Determinización por construcción de subconjuntos	6
5.5. Minimización	7
6. Reglas de prioridad y máxima coincidencia	7
7. Diseño e implementación	7
7.1. Arquitectura de la solución	7
7.2. Formato de salida	7
7.3. Tabla de lexemas	8
8. Tratamiento de errores léxicos	8
9. Plan de pruebas y resultados	8
9.1. Análisis de resultados	9
10. Conclusiones	9
11. Referencias	9
12. Anexos	10

Análisis léxico de un subconjunto de Prolog

Resumen

Se diseñó e implementó un analizador léxico para un subconjunto de Prolog inspirado en SWI-Prolog e ISO Prolog. El lexer recorre el fuente de izquierda a derecha, ignora layout y comentarios, reconoce átomos, variables, números enteros y reales, átomos entre comillas, cadenas, operadores de cláusula, unificación, aritméticos y control, además de delimitadores, conservando línea y columna y registrando errores recuperables. La implementación se realizó en Python 3 sin dependencias externas, aplicando máxima coincidencia y prioridad explícita para operadores compuestos como `:-`, `?-`, `=..`, `==`, `=<`, `>=`, `//` y `**`. Los signos `+` y `-` son operadores y no parte del número. El corpus incluye 20 casos válidos, 8 inválidos y 2 archivos completos. Tras corregir el tratamiento de `REAL+PUNTO`, `tests/valid.pl` produce tokens sin errores y `tests/invalid.pl` reporta 7 errores con posición.

Palabras clave: análisis léxico, Prolog, expresiones regulares, autómatas finitos, máxima coincidencia.

1. Introducción

El análisis léxico es la primera fase de un procesador de lenguajes: agrupa caracteres en tokens y detecta lexemas inválidos antes de cualquier análisis sintáctico. Sin esta fase, errores como literales sin cierre o números mal formados se propagarían a etapas superiores con diagnósticos pobres.

Este informe presenta el subconjunto de Prolog reconocido, su especificación con expresiones regulares, el modelado con AFN/AFD, la determinización y minimización de un caso representativo, las reglas de prioridad, el diseño del lexer en Python, la tabla de lexemas, el tratamiento de errores y el plan de pruebas con resultados.

El alcance termina estrictamente en el léxico: no se valida estructura de cláusulas ni se realiza unificación o análisis semántico.

2. Objetivos

2.1. Objetivo general

Analizar, diseñar, implementar y validar un analizador léxico para un subconjunto definido de Prolog, aplicando alfabetos, lenguajes regulares, expresiones regulares y autómatas finitos.

2.2. Objetivos específicos

- Definir formalmente el subconjunto léxico y una expresión regular por categoría, resolviendo superposiciones.
- Modelar AFN/AFD de las categorías principales y mostrar determinización por subconjuntos y minimización en un caso representativo.

- Implementar en Python un lexer con tipo, lexema, línea, columna, tabla de lexemas y errores recuperables, con corpus reproducible de 20+8 casos y 2 archivos completos.

3. Alcance del analizador

Incluye: átomos simples y entre comillas, variables incluida `_`, enteros y reales con exponente, cadenas con escapes documentados, operadores `:-`, `?-`, `->`, `=`, `\=`, `==`, `\==`, `=.`, `<`, `=<`, `>`, `>=`, `+`, `-`, `*`, `/`, `//`, `**`, `is`, `mod`, `\+`, `!`, `;`, `,`, delimitadores `( ) [ ] { } | .`, comentarios `%` y `/* */`, layout con posición.

Excluye: análisis sintáctico, árboles, validación gramatical, unificación, resolución, ámbitos y semántica. `-12` se tokeniza como `MENOS+ENTERO` por decisión explícita. Fuera de literales/comentarios se restringe a ASCII para mantener el subconjunto verificable.

4. Especificación léxica

Notación: `[A-Z]`, `[a-z]`, `[0-9]`, `ALNUM_=[A-Za-z0-9_]`. Layout y comentarios no generan tokens.

Token	Descripción / ER	Válidos	Inválidos
ATOMO	<code>[a-z][A-Za-z0-9_]*</code> Minúscula inicial	<code>padre, persona_1</code>	<code>Persona, _x, 1abc</code>
VARIABLE	<code>(?:[A-Z] _)[A-Za-z0-9_]*</code> Mayúsc./_ inicial	<code>X, Persona, _Tmp, _</code>	<code>persona</code>
ENTERO	<code>[0-9]+</code>	<code>0, 123</code>	<code>12a</code>
REAL	<code>[0-9]+\.[0-9]+([eE][+-]?[0-9]+)?</code> o <code>[0-9]+[eE][+-]?[0-9]+</code>	<code>3.14, 1.2e-3, 1e10</code>	<code>1., 1.2.3, 7e+</code>
ATOMO_COMILLAS	<code>'([^\n] ...)*'</code> con escapes	<code>'Juan Pérez', 'a+b'</code>	<code>'sin cierre</code>
CADENA	<code>"([^\n] ...)*"</code>	<code>"Hola", "a\nb"</code>	<code>"sin cierre</code>
DOS_PUNTOS_GUION	<code>:-</code>	<code>:-</code>	<code>: solo</code>
CONSULTA	<code>?-</code>	<code>?-</code>	<code>? solo</code>
DCG	<code>-></code>	<code>-></code>	<code>-</code>
IGUAL	<code>=</code>	<code>=</code>	<code>=</code>
DISTINTO	<code>\=</code>	<code>\=</code>	<code>\ solo</code>
IGUAL_ESTRICTO	<code>==</code>	<code>==</code>	<code>= si se exige doble</code>
DISTINTO_ESTRICTO	<code>\==</code>	<code>\==</code>	<code>\= si se exige estricto</code>
DESCOMPONE	<code>=.</code>	<code>=.</code>	<code>=.</code>
MENOR / MAYOR	<code>< / ></code>	<code><, ></code>	<code>—</code>
MENOR_IGUAL	<code>=< (no <=)</code>	<code>=<</code>	<code><=</code>
MAYOR_IGUAL	<code>>=</code>	<code>>=</code>	<code>—</code>
MAS / MENOS / POR / DIV	<code>+ - * /</code>	<code>+ - * /</code>	<code>—</code>
DIV_ENTERA / POTENCIA	<code>// / **</code>	<code>//, **</code>	<code>/ /, * *</code>
IS / MOD	<code>is / mod</code> palabra completa	<code>X is 3</code>	<code>island (es ATOMO)</code>
NO	<code>\+</code>	<code>\+ prueba(X)</code>	<code>\ solo</code>
CORTE / PYC / COMA	<code>! ; ,</code>	<code>! ; ,</code>	<code>—</code>
DELIMS	<code>() [] { } .</code>	<code>() [] { } .</code>	<code>—</code>

Cuadro 1: Catálogo de tokens del subconjunto.

Decisiones: `3.14` es `REAL`; `3.` es `ENTERO+PUNTO`. `island` es `ATOMO` (palabra completa). Escapes admitidos: `\n`, `\t`, `\r`, `\\`, `\'`, `\"`. Otros como `\q` generan error recuperable.

5. Diseño de los autómatas

5.1. AFD para ATOMO y VARIABLE

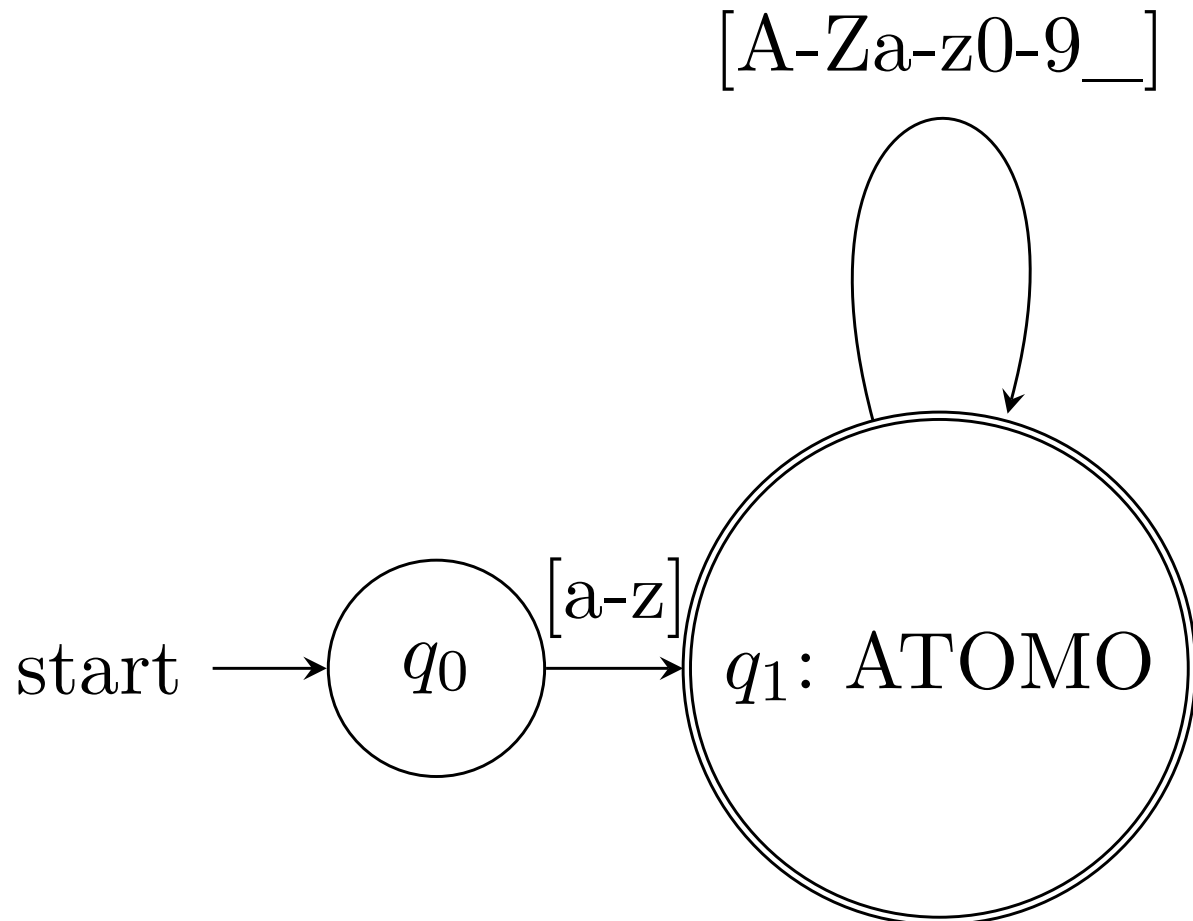


Figura 1: AFD de ATOMO. q_1 acepta; otro carácter termina el token.

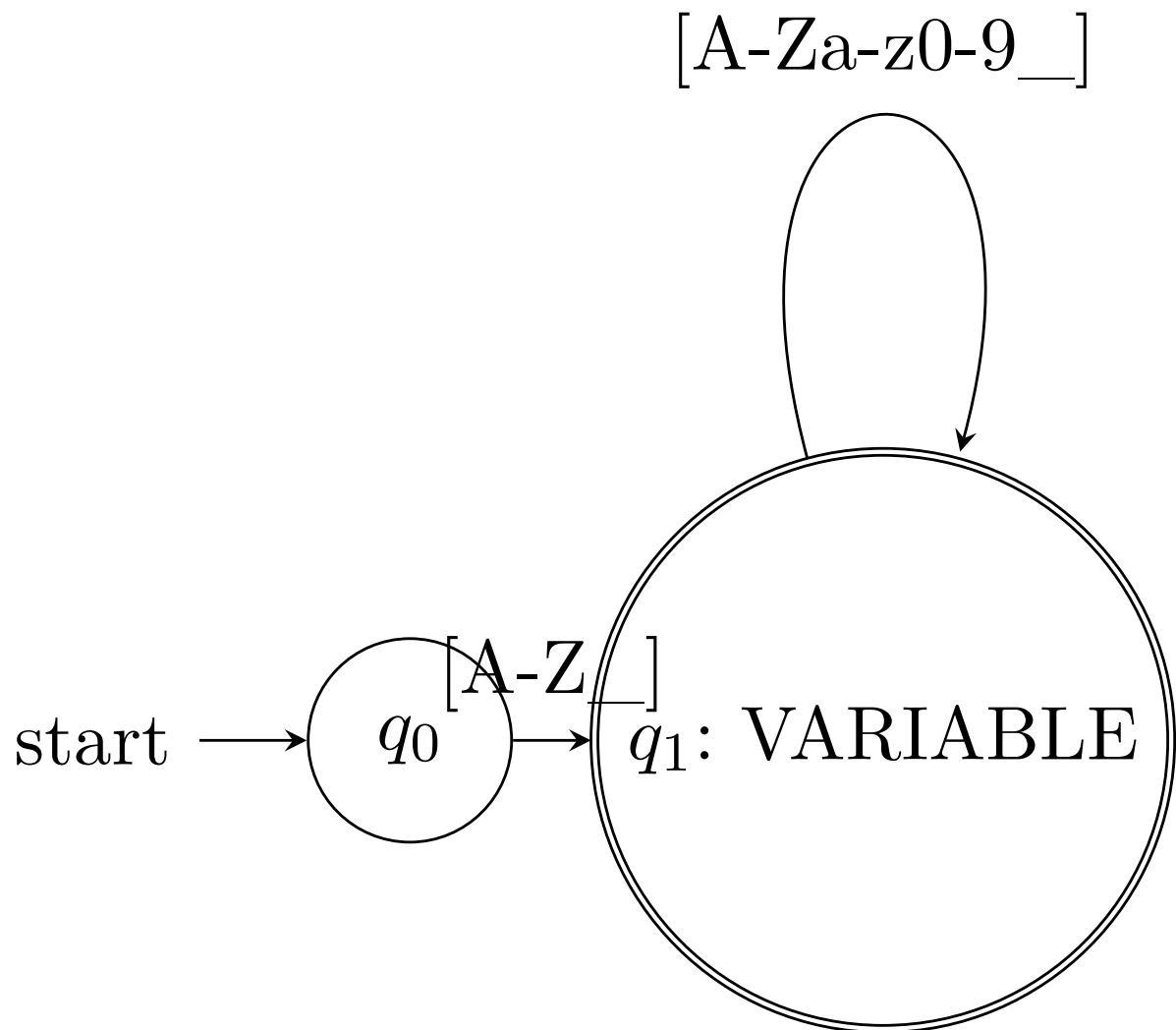


Figura 2: AFD de VARIABLE. _ solo es variable anónima.

5.2. AFD para números

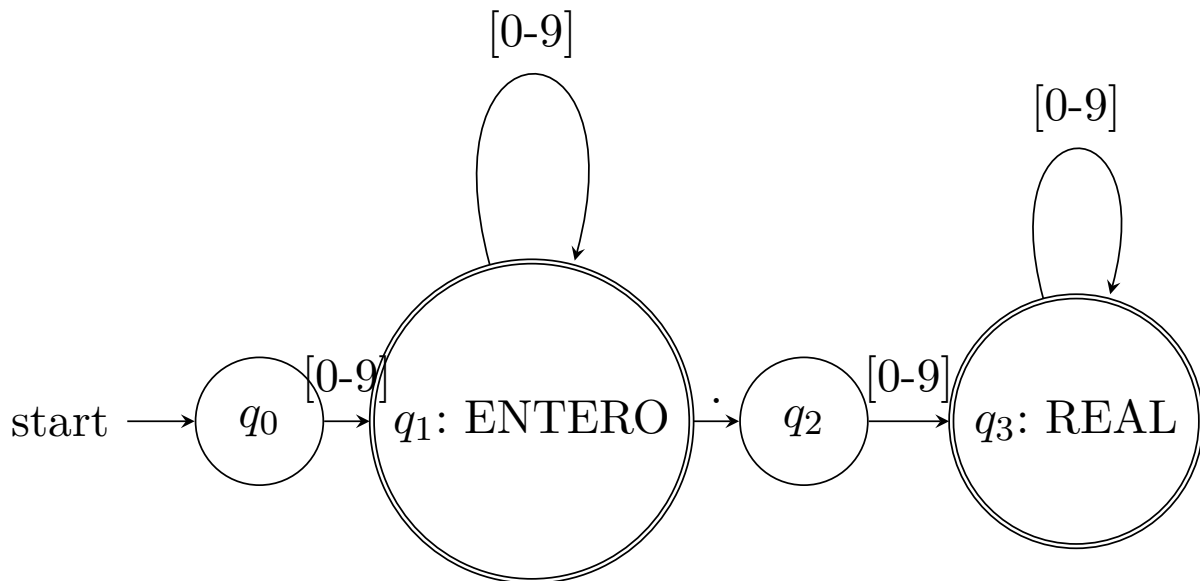


Figura 3: AFD decimal simplificado. La implementación agrega ramas e/E con signo opcional y dígitos desde q_1 y q_3 .

5.3. AFD de operadores con prefijo común

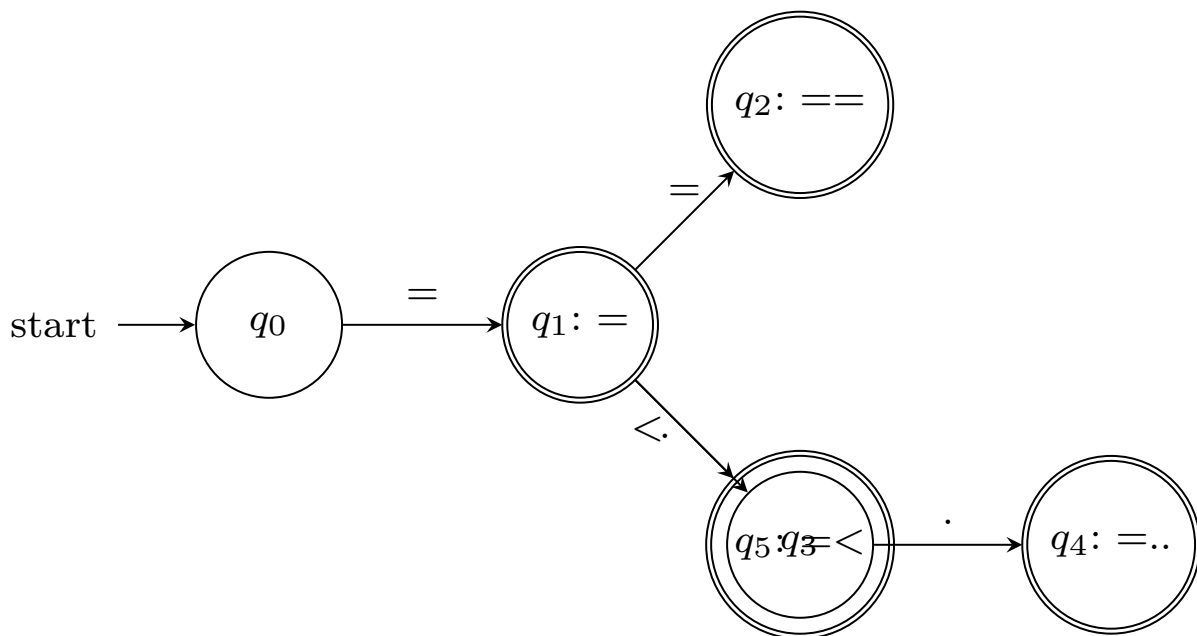


Figura 4: AFD representativo para =, ==, =.., <=. Si q_1 es aceptable pero hay continuación válida, se avanza conservando la última aceptación (máxima coincidencia).

5.4. Determinización por construcción de subconjuntos

AFN representativo para =, ==, <=, =..: $s_0 \xrightarrow{=} s_1$, $s_1 \xrightarrow{=} s_2, s_3, s_4$, $s_2 \xrightarrow{=} s_5$, $s_3 \xrightarrow{<=} s_6$, $s_4 \xrightarrow{<=} s_7$, $s_7 \xrightarrow{<=} s_8$. Aceptación: $s_1: =$, $s_5: ==$, $s_6: <=$, $s_8: =..$

Cierre-ε: D0={s0}; con =: D1={s1,s2,s3,s4}; desde D1 con =: D2={s5}; con <: D3={s6}; con .: D4={s7}; desde D4 con .: D5={s8}. AFD D0...D5 sin no-determinismo, implementable por tabla.

5.5. Minimización

D1=IGUAL, D2=IGUAL_ESTRICTO, D3=MENOR_IGUAL, D5=DESCOMPONE tienen distinto atributo léxico y no pueden fusionarse si el AFD conserva la categoría. Además D1 tiene salidas y es distinguible de terminales. Los estados sin camino a aceptación se agrupan como trampa; la implementación en vez de trampa explícita emite error léxico cuando ningún patrón aplica.

6. Reglas de prioridad y máxima coincidencia

Se consume el lexema más largo; a igual longitud, el orden es: comentarios > literales/identificadores/números > operadores compuestos por longitud > un carácter > delimitadores.

Caso	Regla aplicada	Resultado
\== / \= / ==	Probar \== (3) antes que \= (2)	DISTINTO_ESTRICTO
=.. / =	Probar =.. (3) antes que == (2) y =	DESCOMPONE
:- / :	:- compuesto; : solo es error	DOS_PUNTOS_GUION; : → error
_ / _Var	_ solo es VARIABLE anónima; si sigue alfanum. es una VARIABLE	VARIABLE en ambos, distinto lexema
== / =	Longitud 2 antes que 1	IGUAL_ESTRICTO
//, ** / /, *	Compuestos antes que simples	DIV_ENTERA, POTENCIA
island / is	Palabra completa; si hay letra extra es ATOMO	ATOMO(island) vs IS
3. / 3.14	Real exige dígito tras punto	ENTERO+PUNTO vs REAL

Cuadro 2: Prioridades verificadas en código y tests.

7. Diseño e implementación

7.1. Arquitectura de la solución

Archivo `lexer.py` (~370 líneas, solo `re`, `dataclasses`).
 Clases `Token(tipo,lexema,linea,columna,atributo)` y `ErrorLexico(mensaje,lexema,linea,columna)`.
`Lexer(source)` con `skip_layout_and_comments()`, `read_quoted()`, `read_number()`, `read_identifier()`, bucle `run()` que prueba multi-operadores ordenados por longitud, luego `\+`, simples y delimitadores; si nada coincide, error y avance de 1 para recuperación. `lex_file(path)` y `main()` imprimen tokens y sección ERRORES con `exit 1` si hay errores.

7.2. Formato de salida

<TIPO, 'lexema', linea, columna, atributo>. Fragmento real de `python3 lexer.py tests/valid.pl`:

```

<ATOMO, 'padre', 2, 1, atributo=1>
<PARENTESIS_IZQ, '(', 2, 6>
<ATOMO, 'juan', 2, 7, atributo=2>
<COMA, ',', 2, 11>
<ATOMO, 'ana', 2, 13, atributo=3>
<PARENTESIS_DER, ')', 2, 16>
<PUNTO, '.', 2, 17>
<ATOMO, 'abuelo', 5, 1, atributo=5>
<VARIABLE, 'X', 5, 8, atributo=1>
<DOS_PUNTOS_GUION, ':-', 5, 14>

```

7.3. Tabla de lexemas

Tablas independientes para ATOMO, VARIABLE, CADENA, ATOMO_COMILLAS (dict lexema→índice desde 1). Reaparición reutiliza índice. Ejemplo: padre:1, juan:2, ana:3, pedro:4; variables X:1, Z:2, Y:3. No se resuelven ámbitos ni unificación.

8. Tratamiento de errores léxicos

Error	Ejemplo	Mensaje	Recuperación
Átomo/cadena sin cierre	'sin cerrar	literal sin cierre: "' en 6:7	Reporta apertura, sigue en salto de línea
Comentario sin cierre	/* ...	comentario de bloque sin cierre	Reporta inicio, termina análisis del resto
Carácter no admitido	@, :, ?	carácter no admitido	Avanza 1 y continúa
Número mal formado	123abc, 1.2.3, 7e+	número mal formado	Consume continuación alfanum., reporta, continúa
Escape no admitido	'a\qb'	secuencia de escape no admitida	Reporta, conserva char, continúa

Cuadro 3: Errores con línea, columna y fragmento.

Corrección aplicada: REAL seguido de . solo ya no es error (era bug que marcaba 3.1415. como malformado); solo . + dígito o alfanum. es error. Token \+ unificado como NO.

9. Plan de pruebas y resultados

Corpus en tests/corpus_valid/ (20), tests/corpus_invalid/ (8), tests/valid.pl, tests/invalid.pl, tests/test_lexer.py. Ejecución: python3 tests/test_lexer.py → OK; python3 lexer.py tests/valid.pl exit 0; tests/invalid.pl exit 1 con 7 errores.

ID	Entrada	Objetivo	Esperado	Obtenido	Est.
P01	padre(juan,ana).	Átomo simple	ATOMO...PUNTO sin error	Igual	OK
P02	X, _Tmp, _	Variables	3x VARIABLE	Igual	OK
P03	3.14, 1e10, 1.2e-3.	Reales + punto	REAL+PUNTO	Igual tras fix	OK
P04	=.. == =< >= // ** :- ?-	Máxima coincidencia	8 compuestos	Igual	OK
P05	\== vs \=	Prioridad	Estricto vs simple	Igual	OK
P06	'a+b', "Hola"	Comillas/cadena	2 literales	Igual	OK
P07	is mod island	Palabras op.	IS MOD ATOMO	Igual	OK
P08	'sin, "sin, /* sin	Sin cierre	3 errores	3 errores	OK
P09	123abc, 1.2.3, 7e+, @	Núm/char	4 errores	4 errores	OK
P10	valid.pl / invalid.pl	Archivos	0 vs 7 errores	0 vs 7	OK

Cuadro 4: Plan de pruebas representativo. Corpus completo en repo.

9.1. Análisis de resultados

Se cubren todas las categorías, límites (3. vs 3.14, is vs island), prioridades y errores. Dificultad principal: REAL+PUNTO fin de cláusula se reportaba como malformado; se corrigió exigiendo dígito tras punto para error. Caso límite invalid.pl: el bloque sin cierre ocultaba errores si iba al medio; se movió al final y ahora reporta 7 errores recuperables. Limitación: mal_atom(ABC) no es error léxico (ABC es variable) y números como 123abc. incluyen el punto en el fragmento, aceptable como diagnóstico.

10. Conclusiones

Se logró un lexer que reconoce el subconjunto con posiciones y recuperación, separando léxico de sintaxis. La máxima coincidencia es esencial para compuestos y separar +/- de números simplifica ER. Se cumple el alcance con corpus reproducible y tests. Limitaciones: sin sintaxis/semántica, ASCII fuera de literales, sin anidamiento de bloques. Mejoras: reportes con sugerencia, resaltado, diagramas generados y linter de estilo Prolog.

11. Referencias

- SWI-Prolog. (2024). *SWI-Prolog Syntax*. <https://www.swi-prolog.org/pldoc/man?section=syntax>
- SWI-Prolog. (2024). *The string type and its double quoted syntax*. <https://www.swi-prolog.org/pldoc/man?section=string>
- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson.
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Pearson.

12. Anexos

Recurso	Enlace
Repositorio Git	[https://github.com/eduardoscrs/Tarea-INFO1148-C1]
Código	lexer.py, tests/test_lexer.py
Corpus	tests/corpus_valid/*.pl (20), tests/corpus_invalid/*.pl (8)
Ejecución	python3 lexer.py tests/valid.pl / python3 lexer.py tests/invalid.pl

Anexo A. Evidencias adicionales

Salida de tests/invalid.pl (7 errores tras corrección):

```
[LXICO 6:7] literal sin cierre: ""  
[LXICO 7:6] literal sin cierre: ''  
[LXICO 8:5] nmero mal formado: '123abc.'  
[LXICO 9:5] nmero mal formado: '1.2.3.'  
[LXICO 10:5] nmero mal formado: '7e+.'  
[LXICO 11:1] carcter no admitido: '@'  
[LXICO 13:1] comentario de bloque sin cierre
```